

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)

Artificial Intelligence Application Development

Module 1

Practical –II

Roll no. : T033	Name : Prajapati raj
Class : TYIT	Batch : 1
Date of assignment : 05/08/2026	Date of assignment : 05/08/2026

2. Using a suitable regression dataset, implement Linear Regression using scikit-learn.

Code :

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

✓ 14.4s

data = pd.read_csv("studentmarks.csv")
✓ 0.0s
```

```
X = data[['hours']]
Y = data['marks']
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
)

# Train model
model = LinearRegression()
model.fit(X_train, y_train)

# Display accuracy
print(
    "Training Accuracy:",
    round(model.score(X_train, y_train) * 100, 2)
)

print(
    "Testing Accuracy:",
    round(model.score(X_test, y_test) * 100, 2)
)
```

```
# Predict marks for 11 study hours
new_data = pd.DataFrame([[11]], columns=["hours"])
pred = model.predict(new_data)

print(
    "Predicted Marks for 11 hours:",
    round(pred[0], 2)
)
```

Output :

3. Using the MNIST dataset (digit vs not digit), implement a binary classifier.

Code :

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
```

```
# Load MNIST-like Digits dataset
digits = load_digits()
```

```
# Features and target
X = digits.data
y = digits.target
```

```
# Binary classification
# True if digit is 5, otherwise False
y = (y == 5)
```

```
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
)
```

```
# Train model
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
```

```
# Display accuracy
print("Training Accuracy:",
```

```
round(model.score(X_train, y_train) * 100, 2), "%")

print("Testing Accuracy:",
      round(model.score(X_test, y_test) * 100, 2), "%")

# Predict first test sample
prediction = model.predict([X_test[0]])

if prediction[0]:
    print("Digit is 5")
else:
    print("Digit is not 5")
```

Output :

```
Training Accuracy: 100.0 %
Testing Accuracy: 98.89 %
Digit is not 5
Rajj
```